

Term Project Report (Computer Graphics)

Title: 초차원 알까기 (Hyperdimention Alkkagi)

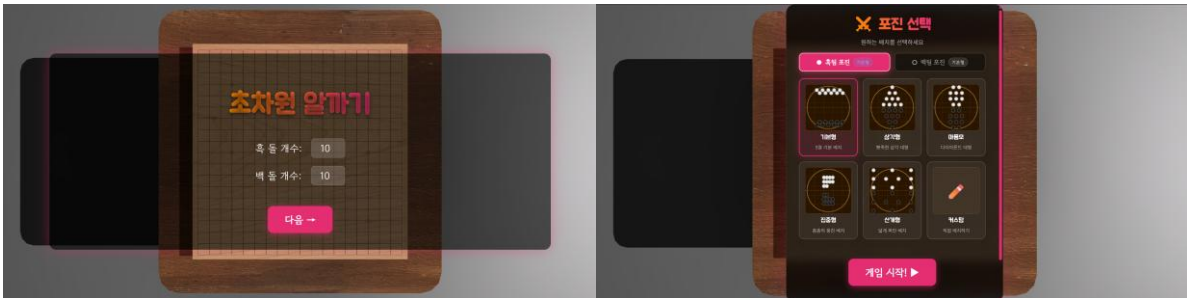
학번: 2024131002 이름: 윤준호

학번: 2016147534 이름: 전정현

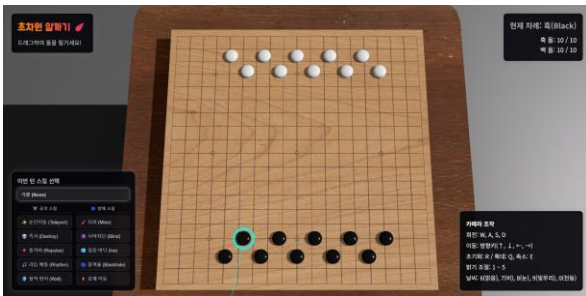
학번: 2022149015 이름: 임은혁

1. 요약: 본 프로젝트 "초차원 알까기(Hyperdimention Alkkagi)"는 3D 그래픽스 이론(Three.js)과 물리 엔진(Rapier3D)을 융합하여 개발한 턴제 하이퍼캐주얼 전략 게임입니다. 단순히 바둑돌을 튕기는 전통적인 알까기 룰에 기초하지만, 그래픽스 수업 시간에 다룬 다양한 렌더링 파이프라인 요소들을 적극 도입하였습니다. 수업에서 배운 조명(Lighting) 및 그림자(Shadow mapping) 이론을 바탕으로 씬(Scene)의 깊이감과 질감을 표현했으며, 블렌더(Blender) Cycles 엔진의 텍스처 베이킹(Texture Bake) 기능을 활용해 오클루전(Occlusion)과 그림자가 정밀하게 반영된 배경 모델을 제작하여 현실감을 극대화했습니다. 카메라 시점 변환(Viewing Pipeline) 및 투영(Projection) 원리를 활용하여 플레이 상황에 맞게 자동으로 줌인/줌아웃 되거나 슬로우 모션이 걸리는 액션 카메라 연출을 구현했습니다. Geometry와 Texture(PBR 기반) 지식을 활용하여 유리 재질감의 바둑돌과 역동적인 배경 환경(비, 눈 등 날씨 파티클)을 렌더링하고, 각 플레이어가 턴마다 벽 설치, 텔레포트, 지뢰 폭발 등 창의적인 스킬을 구사할 수 있게 설계하여 시각적 몰입도와 전략적 깊이를 동시에 극대화한 결과물입니다.

2. 사용법:



1. 게임 진입 및 포진 구성: 시작 화면에서 흑/백 돌 개수를 입력하고, 제공되는 2D 캔버스 UI 상에서 마우스를 통해 초기 진영(포진)을 자유롭게 커스텀하거나 프리셋을 선택하여 게임을 시작합니다. (2D Viewport 매핑 적용)



2. 물리적 드래그 앤 쏘(Drag & Shoot): 본인의 턴에 활성화된 돌을 마우스 좌클릭 후 드래그합니다. 이때 그래픽스 Raycasting 기술이 적용되어 2D 마우스 좌표가 3D 공간 상의 좌표로 역변환(Unprojection)되며, 당기는 거리와 방향에 비례하여 예상 궤적(점선)과 당김선(실선)이 시각적으로 렌더링됩니다. 마우스를 놓으면 충격량(Impulse)이 돌에 가해져 발사됩니다.



3. 동적 뷰잉 파이프라인 제어(Viewing Pipeline): 키보드 W/A/S/D 또는 방향키를 통해 카메라의 구면 좌표계 (Spherical Coordinates)를 실시간으로 제어하여 원하는 각도와 시점으로 씬을 관찰할 수 있습니다. Q/E 키로는 카메라의 줌(시야각 조절)을 수행합니다.



4. 환경 및 스킬 제어: 숫자키 1~5로 Ambient 및 Directional 광원의 강도를 조절(Lighting 세기 변화)하고, 6~0키를 통해 맑음, 비, 눈, 번개 등 다양한 날씨 파티클 시스템을 토글할 수 있습니다. 턴 진행 중 화면 하단의 스킬 UI를 클릭하여 다음 타격 시 발동될 특수 스킬을 지정합니다.

3. 기능 구현 표: (구현 기능 별로 구현 source code 위치를 표기)

	기능	구현위치	비고
1	3D 씬/카메라 생성 및 조명, 그림자 설정 (Lighting, Shadow)	engine.js	강의: 07_Lighting_1/2, 09_DepthTesting
2	구면좌표계 카메라 및 직교/투시 투영 변환 (Viewing, Projection)	cameraManager.js	강의: 05_ViewingPipeline, 05_Projection
3	3D 마우스 픽킹(Raycasting) 및 예상 궤적 라인 렌더링 (Geometry)	interaction.js	강의: 03_Geometry, 04_Transformations
4	3D Mesh 동적 생성 및 PBR 텍스처/재질 매핑 (Texture, Materials)	stone.js, assets.js	강의: 06_Texture
5	물리 엔진 연동을 통한 강체(RigidBody) 트랜스폼 동기화 (Animation)	gameManager.js	강의: 11_KeyframeAnimation, 12_PhysicallyBased
6	동적 파티클 렌더링 및 수명 주기에 따른 점진적 투명도 조절 (VFX)	skillsVFX.js, weather.js	강의: Three.js 실습 (Particles/Sprites)
7	스킬 이벤트 상태 머신 관리 및 계층적 씬 그래프(Scene Graph) 업데이트	SkillManager.js	강의: 13_SceneGraph
8	2D 캔버스 공간 ↔ 3D 월드 공간 간의 좌표 맵핑(Mapping) (커스텀 포진)	formation.js	강의: 04_Transformations, 05_ViewingPipeline
9	조작 및 게임 상태에 따른 2D DOM UI 오버레이 동기화	ui.js	Three.js 응용
10	외부 FBX/GLTF 3D 모델 및 환경 맵 비동기 로딩 파이프라인 설계	main.js, assets.js	강의: 10_Blender, Three.js 실습 (Loading)

4. 기타 사용 기능 (수업시간에 배우지 않은 기능들 및 심화 적용 내역)

4.1 다이내믹 물리 엔진 통합 (Rapier3D)

소스 출처: Rapier 공식 라이브러리 (<https://rapier.rs>)

기능의 이론: 기존 수업에서 다룬 기하학적 변환(Transformations)과 단순 운동학(Kinematics)을 넘어, 질량(Mass), 마찰 계수(Friction), 반발 계수(Restitution)를 기반으로 한 강체 역학(Rigid Body Dynamics) 시뮬레이션입니다. WebAssembly(Wasm) 기반으로 동작하여 매 프레임 발생하는 수많은 충돌 조건(Collision Detection)을 실시간으로 계산하고 충격량 법칙(Impulse-based Contact Mechanics)을 적용합니다.

구현 내용: engine.js에서 물리 세계(physicsWorld)를 구축하고 바둑돌마다 강체와 콜라이더를 결합했습니다. 매 루프마다 물리 엔진이 계산한 3D 변환 행렬(Transformation Matrix)의 위치와 회전 쿼터니언 데이터를 Three.js 메쉬의 position과 quaternion에 강제로 동기화하는 브릿지 로직을 구현했습니다.

4.2 TWEEN.js 기반 보간(Interpolation) 애니메이션

소스 출처: Three.js 내장 팩 및 오픈소스 라이브러리 (<https://github.com/tweenjs/tween.js>)

기능의 이론: 두 상태 사이의 값을 시간에 따라 부드럽게 연결하는 선형 및 비선형 보간(Interpolation) 이론입니다. 갑작스러운 Discrete 변화 대신 파라미터의 시간에 따른 변화율을 수학적 궤적(Easing Functions: Quad, Cubic, Elastic 등)으로 제어하여 시각적 연속성(Visual Continuity)을 달성합니다.

구현 내용: 카메라 시점 이동, 조명 세기(Lighting Intensity) 변경, UI 오버레이의 페이드인/아웃 전환 시 적용했습니다. 리듬 게임 미니게임 페이지 등에서 대상 속성값을 타겟값까지 특정 Easing 함수를 기반으로 실시간 보간 연산하여 부드러운 그래픽스 연출을 완성했습니다. 3. 커스텀 파티클 (VFX) 시스템: 돌이 서로 강하게 충돌하거나 스킬이 발동될 때, 혹은 바둑판 밖으로 떨어질 때(낙사) 발생하는 시각 효과를 위해 수많은 소규모 BoxGeometry 인스턴스를 생성하고 개별적인 수학적 수명(Life)과 중력 벡터를 할당해 폭발/흩날림 이펙트를 자체 구현했습니다.

4.3 커스텀 파티클 (VFX) 시스템

소스 출처: 자체 설계 및 구현 (수업 외 심화)

기능의 이론: 그래픽스 파티클 시스템(Particle Systems) 이론입니다. 하나의 정적 메쉬에 의존하지 않고, 수많은 기하학적 인스턴스를 동적으로 제어합니다. 개별 입자마다 고유한 물리적 속성(초기 속도 벡터, 중력 가속도, 수명 제한)을 부여하고 뉴턴 운동 방정식을 독립적으로 연산하여 가시성을 제어하는 기술입니다.

구현 내용: skillsVFX.js에서 돌의 강한 충돌, 스킬 발동, 바둑판 밖 추락(낙사) 시점에서 소규모 BoxGeometry 배열을 생성했습니다. 매 프레임 델타 타임에 비례하여 중력 벡터를 더해 위치를 갱신하고, 수명(Life) 감소에 따라 투명도(Opacity)를 선형 감쇠시켜 자동 소멸하는 고성능 폭발/흩날림 이펙트를 자체 연산 로직으로 구현했습니다.

4.4 고도화된 액션 줌 카메라

소스 출처: 자체 설계 및 구현 (수업 외 심화)

기능의 이론: 뷰잉 파이프라인(Viewing Pipeline) 및 카메라 추적 상태 기계(Camera Tracking State Machine) 이론입니다. 3D 공간 상의 가변적인 두 오브젝트 간의 거리를 기하학적으로 계산하고, 두 벡터의 Midpoint(중심점)를 카메라 뷰 행렬(View Matrix)의 LookAt 타겟으로 실시간 갱신하는 동적 카메라 제어 기법입니다.

구현 내용: 돌들이 고속으로 충돌하기 직전 상태 기계(gameState)를 감지하여 타임스케일(Delta Time)을 강제로 낮춰 슬로우 모션(Slow Motion) 효과를 발동시켰습니다. 충돌 직전 두 돌의 위치 벡터 평균값을 계산해 카메라의 위치와 시선을 해당 중심점으로 Lerp 보간하며 강렬한 타격 줌인/줌아웃 시퀀스를 연출했습니다.

4.5 블렌더 모델 텍스처 제작

소스 출처: Blender 공식 소프트웨어 패키지 (<https://www.blender.org>)

기능의 이론: 글로벌 일루미네이션(Global Illumination) 및 광선 추적(Ray Tracing) 기반의 텍스처 베이킹(Texture Baking) 이론입니다. 정적인 배경 오브젝트의 렌더링 비용을 줄이기 위해, 고해상도 렌더 엔진의 광선 연산 결과물인 정적 그림자(Static Shadows)와 앰비언트 오클루전(Ambient Occlusion, 주변광 차폐) 데이터를 2D 텍스처 이미지의 UV 맵 공간에 미리 구워내는(Baking) 최적화 기법입니다.

구현 내용: 블렌더 기본 조작을 통해 방, 책상, 바둑판 등의 배경 에셋을 제작한 후 Cycles 렌더 엔진의 Texture Bake 기능을 실행했습니다. 이를 통해 복잡한 실시간 연산 없이도 정밀한 명암과 구석진 곳의 어두운 음영이 미리 반영된 고품질 PBR 기반 배경 텍스처를 완성하여 Three.js 환경에 적용했습니다.

4.6 셰이더를 통한 이펙트 제어

소스 출처: 자체 개발 GLSL 커스텀 셰이더 (WebGL Shader Specification)

기능의 이론: 프로그래머블 렌더링 파이프라인(Programmable Rendering Pipeline)의 프래그먼트 셰이더(Fragment Shader) 이론입니다. CPU 연산을 거치지 않고 GPU 단에서 직접 픽셀의 UV 좌표와 유니폼(Uniform) 변수를 연산하여 수학적 파형 함수(Waveform Functions: Sine, Exponential Decay)를 통해 표면의 색상과 투명도를 실시간 제어하는 기술입니다.

구현 내용: WallSkillVFX 디렉토리 하위의 shVert.glsl과 shFrag.glsl 코드를 설계했습니다. 물리 엔진으로부터 바둑돌과 장벽의 실제 충돌 지점 좌표를 역계산하여 셰이더의 Uniform 변수(uHitPosition)로 전달하고, 경과 시간(uTime) 변수를 파동 방정식에 대입하여 장벽 표면에서 동심원 형태로 파형이 퍼져나가는 전자기적 충격파 이펙트를 완전 동적으로 구현했습니다.

5. 기타 (창의성, 차별성, 심미성 등)

[창의성 및 차별성]

기존 알카기라는 고전 게임 규칙의 틀을 깨고 "공간의 지배(텔레포트, 장벽)"와 "타이밍(슬로우 모션, 리듬 게임 요소)"이라는 전략적 레이어를 창의적으로 더했습니다. 예를 들어, 상대방의 경로를 차단하는 물리적 Wall(장벽)을 씬 그래프(Scene Graph)에 실시간 삽입하고, GLSL 커스텀 셰이더를 통해 충돌 지점의 UV 좌표와 시간에 따라 장벽에서 충격파가 퍼지는 효과를 직접 구현하거나, 충돌 순간 스킬이 폭발하는 시스템은 단순 보드게임을 턴제 액션 전략 게임으로 탈바꿈시킵니다. 물리 엔진의 궤적 예측 Raycast를 통해 당구의 스네이크 라인처럼 미리 충돌을 예상하고 스킬을 조합하는 플레이 경험은 이 게임만의 강력한 차별점입니다.

[심미성 및 렌더링 완성도]

수업 내용 중 Texture 및 Lighting(조명) 이론을 적극 응용하여 극적인 심미성을 달성했습니다. 바둑판과 바둑돌에 PBR(Physically Based Rendering) 재질의 거칠기(Roughness)와 코팅(Clearcoat) 속성을 튜닝하여 사실적인 반사와 광택을 연출했습니다. 이에 더해 블렌더(Blender) Cycles 엔진의 텍스처 베이킹(Texture Bake) 기능을 활용해 오클루전(Occlusion)과 그림자가 정밀하게 반영된 배경 모델 텍스처를 직접 제작 및 적용하여 씬의 현실감을 한층 끌어올렸습니다. 또한, 고해상도 그림자 맵(Shadow Mapping, 4096 해상도)과 PCFSoftShadow 필터링을 적용해 부드러운 그림자 윤곽선을 만들어냈으며, 배경에는 실시간으로 기하학적 파티클이 쏟아지는 비와 눈을 구현해 차갑고도 세련된 초차원적 분위기를 완성했습니다.